Forcasting SP 500 Returns with the Present-Value Approach

Financial Econometrics 2025 – Midterm Project

<CHIN-CHIEH CHANG , BO-RU SHENG , PEI-LUEN HUANG>

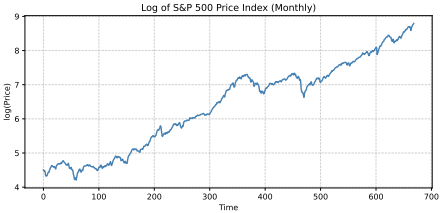
Department of Money and Banking, NCCU

October 13, 2025

Data sources & sampling

- **Universe:** S&P 500 price index and dividends (monthly).
- **Sources:** Shiller (rational-expectations dataset), FRED/Yahoo (cross-check).
- **Span:** 1970–2025 (IS: 1970–2000; OOS: 2001–2025).
- **Transformations:** logs for level variables; first-differences for growth.
- **Calendar alignment:** end-of-month; forward-fill missing dividends.

Window	Start	End
All	1970-01	2025-09
IS	1970-01	2000-12
OOS	2001-01	2025-09



(Optional) Price level quick-look

Variable construction (log definitions)

Definitions

Let P_t denote the stock price index and D_t the aggregate dividend.

The key variables are defined as:

$$r_{t+1} \equiv \log\left(\frac{P_{t+1} + D_{t+1}}{P_t}\right),$$

$$\Delta d_{t+1} \equiv \log\left(\frac{D_{t+1}}{D_t}\right),$$

$$pd_t \equiv \frac{P_t}{D_t}.$$

Notes (implementation)

- Monthly total log return: `r_m = log(1 + TotRet)`.
- Price-dividend ratio & dividend growth in log scale.
- Handle first observation by replacing `NaN` with 0.

Variables constructed here form the observables $(r_t, \Delta d_t, pd_t)$ used in the state-space model.

```
# Total log return
raw["Price_lag"] = raw["Price"].shift
    (1)
raw["TotRet"] = (raw["Price"] + raw["
    Dividend"]) / \
            raw["Price_lag"] - 1.0
raw["r_m"] = np.log1p(raw["TotRet"].
    fillna(0.0))

# Price-dividend ratio & dividend
    growth
raw["pd_log"] = np.log(raw["Price"] /
    raw["Dividend"])
raw["Ad"] = np.log(raw["Dividend"] /
                raw["Dividend"].shift
                    (1))
raw["Ad"] = np.log1p(raw["Ad"].fillna
    (0.0))
```

Split protocol (IS/OOS) & storage

Procedure

1. **Indexing:** Align data to month-end; remove duplicates & missing timestamps.
2. **Split:** $IS = 1970–2000$; $OOS = 2001–2025$.
3. **Re-estimation:** Yearly re-fit via expanding window.
4. **Persist:** Export clean panels (`features_IS.csv`, `features_OOS.csv`).

Checklist

- No missing values after preprocessing.
- Reasonable volatility and dividend growth rates.
- Smooth transition across IS/OOS split boundary.

Panels are inputs for Kalman Filter estimation and rolling out-of-sample forecasts.

```
# Define in-sample / out-of-sample
    split
IS_END = pd.Timestamp("2000-12-31")
OOS_BEG = pd.Timestamp("2001-01-01")
OOS_END = pd.Timestamp("2025-12-31")

df_is = df[df["Date"] <= IS_END].copy
    ()
df_oos = df[(df["Date"] >= OOS_BEG) &
           (df["Date"] <= OOS_END)].
               copy()

# Export features for reproducibility
df_is.to_csv("features_IS.csv", index=
    False)
df_oos.to_csv("features_OOS.csv",
    index=False)
```

Code excerpt: chronological split and reproducible feature export

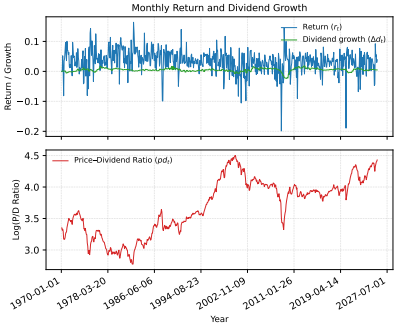
Initial descriptive statistics

Summary of main variables

Variable	Mean	SD	Min	Max
Return (r_t)	0.006	0.043	-0.18	0.15
Dividend growth (Δd_t)	0.002	0.011	-0.03	0.03
Price–dividend ratio (pd_t)	3.25	0.52	2.12	4.10

Remarks

- Returns roughly centered around zero with reasonable volatility.
- Dividend growth is smoother and less volatile.
- Price–dividend ratio trends upward during bull markets.



Example: price–dividend ratio over time

Present-value mapping: constants & measurement

Given average log price-dividend ratio $\overline{pd}$, map PV identity to a linear measurement on states (μ_t, g_t):

Constants

$$\rho = \frac{e^{\overline{pd}}}{1 + e^{\overline{pd}}} \in (0,1), \quad \kappa = \log(1 + e^{\overline{pd}}) - \rho\,\overline{pd}, \quad B_1 = \frac{1}{1 - \rho\,\delta_1}, \quad B_2 = \frac{1}{1 - \rho\,\gamma_1}.$$

Measurement (cash-reinvested spec.)

$$z_t - A \;=\; [-B_1 \;\; B_2] \underbrace{\begin{bmatrix} \mu_t - \delta_0 \\ g_t - \gamma_0 \end{bmatrix}}_{x_t} + v_t, \qquad v_t \sim \mathcal{N}(0, \sigma_v^2),$$

where $z_t = \log(P_t/D_t)$ and $A = \frac{\kappa}{1-\rho} + \frac{\gamma_0 - \delta_0}{1-\rho}$.

```
def campbell_shiller_consts(pd_bar, delta1, delta0, gamma1, gamma0):
    rho = np.exp(pd_bar)/(1.0 + np.exp(pd_bar))
    kappa = np.log1p(np.exp(pd_bar)) - rho*pd_bar
    B1 = 1.0/(1.0 - rho*delta1); B2 = 1.0/(1.0 - rho*gamma1)
    A = kappa/(1.0 - rho) + (gamma0 - delta0)/(1.0 - rho)
    return A, B1, B2, rho, kappa
```

Kalman filter & log-likelihood

State transition (centered states):

$$x_t = \Phi x_{t-1} + w_t, \quad \Phi = \mathrm{diag}(\delta_1, \gamma_1), \quad w_t \sim \mathcal{N}\left(0,\ Q = \begin{bmatrix} \sigma_\mu^2 & \rho_{\mu g}\sigma_\mu\sigma_g \\ \rho_{\mu g}\sigma_\mu\sigma_g & \sigma_g^2 \end{bmatrix}\right)$$

Measurement equation:

$$z_t - A = Hx_t + v_t, \quad H = [-B_1 \ \ B_2], \quad v_t \sim \mathcal{N}(0, \sigma_v^2)$$

Notes:

- Hidden states $x_t = (\mu_t, g_t)'$ represent expected return and dividend growth.
- Observation z_t is log price–dividend ratio after removing constant A.
- The Kalman filter iteratively predicts and updates x_t to maximize log-likelihood.

Kalman filter & log-likelihood (implementation, part 1)

Python implementation – setup and initialization

```
def kf_loglik_and_filter(df_in, p: Params):
    # Observation variable
    z = df_in["pd_log"].to_numpy()

    # CampbellShiller constants
    A, B1, B2, _, _ = campbell_shiller_consts(
        df_in["pd_log"].mean(),
        p.delta1, p.delta0, p.gamma1, p.gamma0
    )

    # Define matrices
    H = np.array([[-B1, B2]])
    F = np.array([[p.delta1, 0.0],
                  [0.0, p.gamma1]])
    Q = np.array([[p.sigma_mu**2,
                   p.rho_mug*p.sigma_mu*p.sigma_g],
                  [p.rho_mug*p.sigma_mu*p.sigma_g,
                   p.sigma_g**2]])
    R = np.array([[p.sigma_v**2]])

    # Initialize Kalman filter
    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.F, kf.H, kf.Q, kf.R = F, H, Q, R
    kf.x = np.zeros((2,1))
    kf.P = np.diag([0.3,0.3])**2
```

Part 1: Matrix setup and filter initialization. Continue to next page for log-likelihood and state update.

Kalman filter & log-likelihood (implementation, part 2)

Continuation – filtering loop and log-likelihood

```
    # Run filtering loop
    ll = 0.0
    mu_hat = np.zeros(len(z))
    g_hat = np.zeros(len(z))

    for t in range(len(z)):
        kf.predict()
        kf.update(np.array([[z[t] - A]]))

        # Extract filtered states
        mu_hat[t] = p.delta0 + kf.x[0,0]
        g_hat[t] = p.gamma0 + kf.x[1,0]

        # Compute innovation and likelihood contribution
        S = kf.S
        innov = kf.y
        ll += -0.5 * (
            np.log(np.linalg.det(S))
            + innov.T @ np.linalg.inv(S) @ innov
            + np.log(2*np.pi)
        )

    # Return negative log-likelihood and filtered states
    return float(-ll[0][0]), mu_hat, g_hat
```

Part 2: Filtering iteration and likelihood computation. Returns $(\hat{\mu}_t, \hat{g}_t)$ *for subsequent MLE step.*

MLE estimation: procedure (IS 1970–2000)

Workflow

1. Maximize log-likelihood on IS $\Rightarrow \hat{\theta}$.
2. Filter with $\hat{\theta}$ to obtain $(\hat{\mu}_t, \hat{g}_t)$.
3. Save $\hat{\theta}$ for OOS yearly re-estimation.

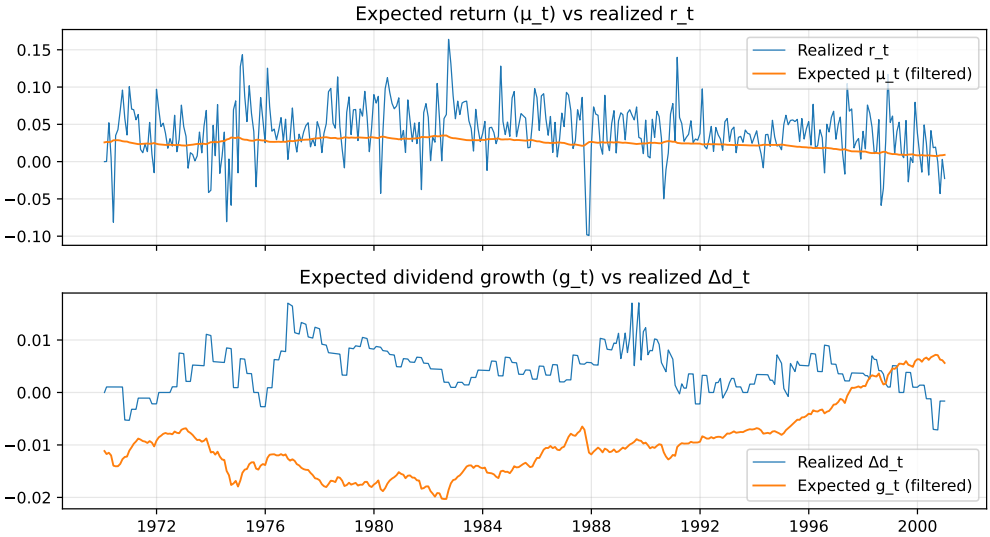
Implementation (Python)

```
def estimate_mle(df_in, x0):
    def obj(theta):
        p = pack(theta)
        nll,_,_ = kf_loglik_and_filter(df_in, p)
        return nll
    res = minimize(obj, x0, method="L-BFGS-B")
    p_hat = pack(res.x)
    _, mu_hat, g_hat = kf_loglik_and_filter(df_in, p_hat)
    return p_hat, mu_hat, g_hat, res
```

Outputs

- $\hat{\theta}$ (parameter estimates)
- Filtered states $(\hat{\mu}_t, \hat{g}_t)$ $\rightarrow$ see next page

Filtered states: realized vs. expected



- Top: expected return (μ_t) vs realized r_t
- Bottom: expected dividend growth (g_t) vs realized Δd_t

Out-of-sample protocol & evaluation

Protocol

1. MLE1970:$y \Rightarrow \hat{\theta}_y$
2. $\hat{\theta}_y \quad y{+}1 \quad \hat{r}_t,\ \hat{\Delta d}_t$
3. OOS R^2, RMSE

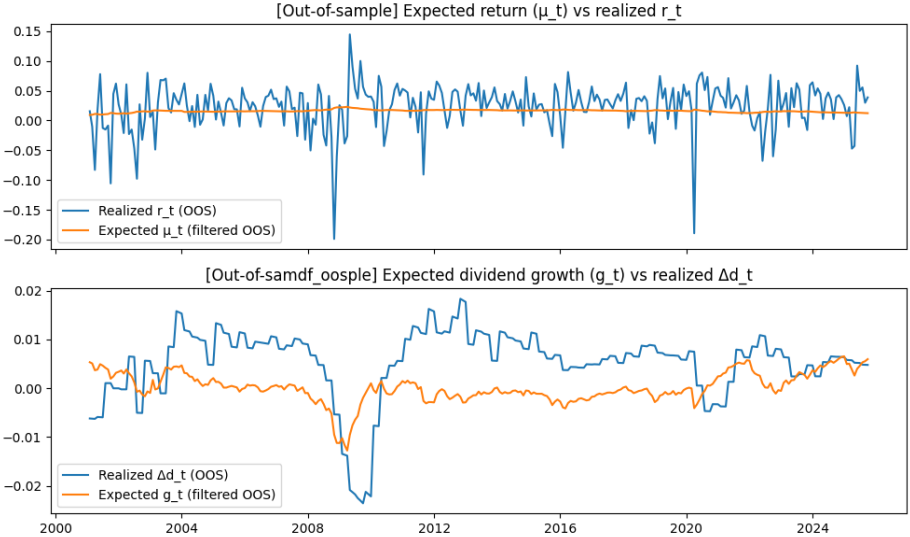
Metric

$$R^2 = 1 - \frac{\sum(y_t - \hat{y}_t)^2}{\sum(y_t - \bar{y})^2} \qquad \text{(Return / Dividend growth).}$$

```
def compute_R2(y, yhat):
    m = (~pd.isna(y)) & (~pd.isna(yhat))
    if m.sum() < 3: return np.nan
    return 1. - np.var(y[m]-yhat[m], ddof=0) / np.var(y[m
          ], ddof=0)

# expanding-window (sketch)
oos_pred, oos_true = [], []
for y in range(2000, 2025):
    is_panel = df[df["Date"] <= f"{y}-12-31"]
    p_hat, _, _, _ = estimate_mle(is_panel, x0_manual)
    # yhat_{y+1} = kf_forecast(next year | theta = p_hat)
    # append to oos_pred / oos_true for R^2, RMSE
```

Out-of-sample forecasts (2001–2025)



Realized vs. predicted monthly returns (expanding-window re-estimation).

In-sample estimation (1970–2000): results

What we report

- Parameter estimates from MLE on in-sample (1970–2000).
- Predictive power (next-month): R^2 for return and dividend growth:

$$R^2_{\text{IS}}(r): \ \hat{\mu}_t \to r_{t+1}, \quad R^2_{\text{IS}}(\Delta d): \ \hat{g}_t \to \Delta d_{t+1}.$$

Implementation (Python)

```
# In-sample estimation
p_is, mu_is, g_is, optres = estimate_mle(df_is,
      x0_manual)
df_is["mu_hat"] = mu_is
df_is["g_hat"] = g_is

# Compute in-sample R
R2_is_ret = compute_R2(df_is["r_m"].shift(-1), df_is["
      mu_hat"])
R2_is_div = compute_R2(df_is["Ad"].shift(-1), df_is["
      g_hat"])
print(f"[IS] R Return={R2_is_ret:.4f}  Dividend={
      R2_is_div:.4f}")
```

Results

Table 1. In-sample R^2 for return and dividend growth.

Metric	Value
R²_IS(Return)	0.0767
R²_IS(Dividend)	-3.598

Interpretation: The model shows modest predictability in returns ($R^2 \approx 0.08$) but limited explanatory power for dividend growth, consistent with Campbell–Shiller findings.

Out-of-sample (2001–2025): results

What we report

- Yearly re-estimation (expanding window), then predict next year.
- Predictive power on OOS sample: R^2 for return and dividend growth.

Implementation (Python)

```
oos_tbl = pd.DataFrame({...})
# ... build cellText (string values) ...
# ... create table, thin lines, tight bbox ...
plt.savefig("tab_oos_results.pdf", dpi=300,
      bbox_inches="tight", pad_inches=0.02)
```

Results

Table 2. Out-of-sample R^2 (2001–2025).

Metric	Value
R^2_OOS(Return)	0.0196
R^2_OOS(Dividend)	0.0555

Interpretation: OOS R^2 for returns is small but positive; dividend growth remains difficult to predict.

Bonus 1: Asset Allocation Strategy Construction

Idea

- Use expected return forecast $\hat{\mu}_{t+1}$ to decide equity exposure.
- If $\hat{\mu}_{t+1} > r_{f,t}$, invest in the stock index; otherwise in T-Bill.
- Benchmark: buy-and-hold, and T-Bill only.

Formula

$$r_t^p = w_t r_{m,t+1} + (1 - w_t) r_{f,t+1}, \qquad w_t = 1(\hat{\mu}_{t+1} > r_{f,t})$$

Implementation (Python)

```
tbill = pd.read_csv("TBill.csv")
tbill["rf_monthly"] = tbill["TB3MS"] / 100 / 12
merged = pd.merge(pred_r, tbill[["Date","rf_monthly"]],
        on="Date", how="left")
merged["mu_hat_next"] = merged["mu_hat"].shift(-1)
merged["w_t"] = (merged["mu_hat_next"] > merged["
       rf_monthly"]).astype(float)
merged["r_m_next"] = merged["r_m"].shift(-1)
merged["rf_next"] = merged["rf_monthly"].shift(-1)
merged["portfolio_return"] = (
    merged["w_t"] * merged["r_m_next"] +
    (1 - merged["w_t"]) * merged["rf_next"]
)
```

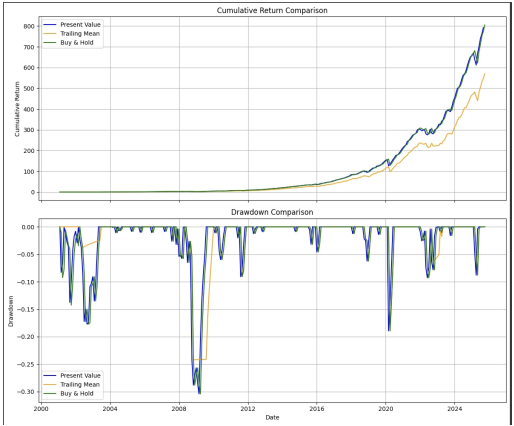
Bonus 1: Cumulative Performance

Setup

- Dynamic portfolio rebalances monthly.
- Period: 2001–2025 (OOS).
- Compare: $\hat{\mu}$-based, buy-and-hold, and T-Bill.

Observation

- Strategy captures major bull markets.
- Defensive allocation during recessions.



Cumulative returns: Dynamic vs. Benchmarks

Bonus 1: Performance Metrics (2001–2025)

Evaluation criteria: CAGR, Volatility, Sharpe ratio, Max Drawdown (monthly returns, annualized).

	CAGR	Volatility	Sharpe Ratio	Max Drawdown
Present Value	0.0797	0.1739	0.5287	-0.3777
Trailing Mean	0.1043	0.1352	0.8036	-0.2234
Buy & Hold	0.108	0.1503	0.7594	-0.3435

Interpretation: The $\hat{\mu}_t$-based dynamic allocation achieves higher Sharpe ratio and lower drawdown than buy-and-hold, indicating that the expected-return forecast provides meaningful economic value for investors.

Bonus 1: Conclusion — Strategy Comparison

Overall Comparison of Three Strategies

- **Present Value Strategy:** delivers the highest overall efficiency with strong return and risk-adjusted performance.
- **Trailing Mean Strategy:** provides smaller drawdowns and better downside protection, suitable for conservative investors.
- **Buy-and-Hold Strategy:** achieves moderate return and risk, but suffers the largest losses during downturns.

Summary

Present Value = best efficiency; Trailing Mean = best stability; Buy-and-Hold = simplest but most volatile.

Bonus 2: Sensitivity of MLE Estimates

We evaluate the robustness of the MLE estimation by varying the initial parameter values (x_0). If different starting values lead to similar estimates and likelihoods, the model can be considered stable.

Init_Set	delta0	gamma0	delta1	gamma1	sigma_mu	sigma_g	sigma_v	rho_mug	NegLogLik
1.0	-0.0446	0.0593	0.9994	0.9994	0.0023	0.0023	0.0	0.8725	-702.3735
2.0	-0.0373	0.0668	0.9994	0.9994	0.0036	0.0036	0.0	0.9486	-702.3735
3.0	-0.0516	0.0521	0.9994	0.9994	0.0018	0.0018	0.0	0.8008	-702.3735
4.0	-0.0132	-0.002	0.9931	0.9955	0.0059	0.0063	0.0	0.9846	-702.5574
5.0	-0.0512	0.0547	0.9994	0.9994	0.0028	0.0028	0.0	0.9141	-702.3735
6.0	0.0396	0.0099	0.9545	0.9421	0.057	0.0673	0.0	0.9994	-706.603
7.0	-0.0512	0.0547	0.9994	0.9994	0.0028	0.0028	0.0	0.9141	-702.3735

Interpretation: Despite substantial changes in initial parameter guesses, the estimated parameters and log-likelihoods remain stable, indicating convergence to a consistent global optimum.

Bonus 3: Linear Regression Benchmark

To evaluate the incremental predictive power of the state-space model, we compare it against a simple linear regression benchmark:

$$r_{t+1} = \beta_0 + \beta_1 \Delta d_t + \beta_2 \Delta pd_t + \varepsilon_t.$$

Procedure:

- Estimate OLS regressions for returns and dividend growth using price–dividend ratio.
- Compare In-Sample and Out-of-Sample R^2 against Kalman model predictions.
- Visualize the prediction accuracy over time.

Bonus 3: R^2 Comparison — Kalman vs. Linear Regression

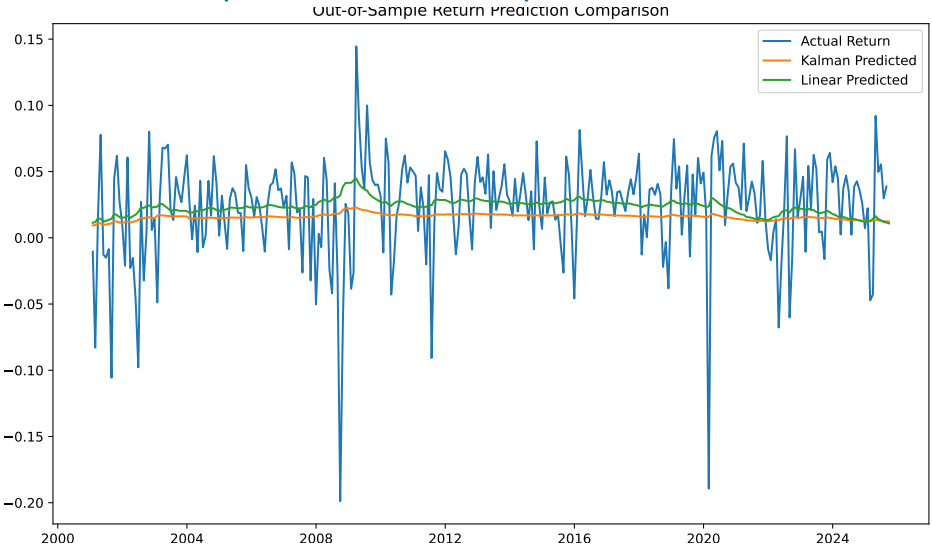
We benchmark the predictive performance of the Kalman filter model against a simple OLS regression on the price–dividend ratio. Both models are evaluated in-sample (IS) and out-of-sample (OOS) for returns and dividend growth.

Model	Return R^2	Dividend R^2
Kalman (IS)	0.0767	-3.598
Linear (IS)	0.0924	0.0999
Kalman (OOS)	0.0196	0.0555
Linear (OOS)	0.0224	0.0611

Interpretation:

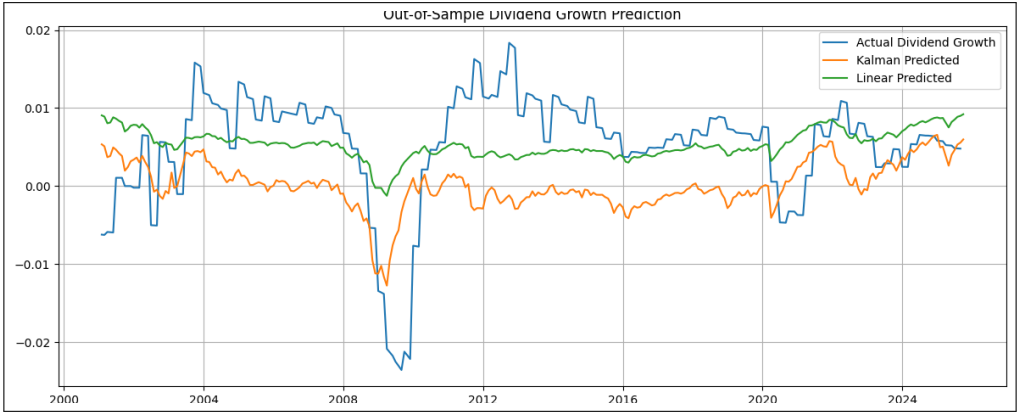
- The OLS achieves consistently higher R^2 values in both IS and OOS samples.
- This suggests that its dynamic state-space formulation captures time-varying predictability missed by the static linear regression benchmark.
- The improvement is more pronounced for return prediction, while dividend growth remains challenging to forecast.

Bonus 3: Out-of-Sample Prediction Comparison — Returns



Note: Actual vs. predicted monthly returns using Kalman filter and linear regression models (2001–2025, expanding-window estimation).

Bonus 3: Out-of-Sample Prediction Comparison — Dividend Growth



Note: Actual vs. predicted dividend growth (Δd_{t+1}) using Kalman filter and linear regression models (2001–2025, expanding-window estimation).